



FPGA based Hardware Scheduling approach for optimising DSP pipelines

Connor Sinclair

This thesis is presented as part of the requirements for the conferral of the degree:

Bachelor of Software Engineering (Hons.)

Supervisor:
Dr. Philip Leong

The University of Sydney
School of Engineering

2025

Declaration

I, *Connor Sinclair*, declare that this thesis is submitted in partial fulfilment of the requirements for the conferral of the degree *Bachelor of Software Engineering (Hons.)*, from the University of Sydney, is wholly my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualifications at any other academic institution.

Connor Sinclair

August 23, 2025

Abstract

Field-programmable gate arrays (FPGAs) offer unprecedented parallelism and low latency for real-time audio processing. However, FPGA toolchains typically perform static compilation and mapping of fixed designs. In practice, audio signals and user-defined module graphs vary over time, presenting an opportunity to predictively reconfigure the FPGA fabric. In this thesis, I propose an operator-theoretic and state-space analysis framework that models DSP modules as dynamical systems, using observables and custom signal correlation metrics to forecast workload and adjust resource allocation. Unlike conventional flows, this approach can anticipate the evolving demands of a modular audio effect chain (including nonlinear modules) and remap kernels (e.g. heavy multipliers or trigonometric units) onto LUTs, DSP slices, or BRAMs in accordance to resource demands. Expected contributions include: (1) an analytical model of audio DSP modules grounded in operator theory (Koopman embeddings) and state-space representations; (2) coherent metrics of signal interaction for resource tuning; and (3) a prototype toolchain enabling runtime FPGA optimization in an accessible form. This work bridges a gap between control-theoretic modeling and FPGA architecture, offering audio engineers and embedded developers a more adaptive FPGA DSP platform that can improve throughput, support complex nonlinear & stochastic audio effects and enable massive real-time parallelisation.

Acknowledgements

Contents

Abstract	iii
Acknowledgements	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Motivation	1
1.2 Aims	1
1.3 Contributions	1
1.4 Thesis Structure	1
2 Literature review	2
3 Combined CSP & DSP approach to categorising signal families	3
3.1 Cyclostationary analysis	3
3.1.1 Relation between Strict Periodicity and Cyclostationarity . . .	4
3.2 Novel combined CSP, DSP approach to creating FPGA cache heirarchies	5
3.2.1 Procedure to storing arbitrary signals	7
3.3 Synthesizable construction of the recovery method, T , for FPGA's . .	7
4 Architectural Design	10
4.1 Filter design	10
4.1.1 Procedure for approximating Error statistics on filter kernels .	10
5 Control theoretic model for dynamic FPGA resource balancing	13
A Appendix	14

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

1.2 Aims

1.3 Contributions

1.4 Thesis Structure

Chapter 2

Literature review

Chapter 3

Combined CSP & DSP approach to categorising signal families

3.1 Cyclostationary analysis

A signal with the property $x(t) = x(t + T) \forall t, T > 0$ is called *strictly periodic* (or *periodic pointwise*). A composition of periodic signals $h(t) = \sum_{i=0}^N x_i(t)$ is also periodic $\iff \frac{T_i}{T_j} \in \mathbb{Q} \forall i, j$ (i.e., the periods are commensurable). The period of $h(t)$ can therefore be described as

$$T = \text{lcm}(T_0, T_1, \dots, T_N).$$

A *wide-sense cyclostationary* (WSC) signal is defined by

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t + T)], \quad R_x(t, \tau) = R_x(t + T, \tau),$$

where $R_x(t, \tau) = \mathbb{E}[x(t)x^*(t + \tau)]$. That is, both the mean $\bar{x}(t)$ and the autocorrelation function $R_x(t, \tau)$ are periodic in t . More generally, a signal is cyclostationary *iff* there exists some n th-order nonlinear transformation of $x(t)$ that produces a finite additive sinusoidal component [**<empty citation>**].

The definition of the *spectral parameter* [**<empty citation>**] is

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t) e^{-j2\pi\alpha t} dt.$$

For $x(t) = a \cos(2\pi\beta t + \theta)$:

$$x(t) = \frac{a}{2} (e^{j(2\pi\beta t + \theta)} + e^{-j(2\pi\beta t + \theta)}).$$

Substituting into the definition:

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{a}{2T} \left[e^{j\theta} \int_{-T/2}^{T/2} e^{j2\pi(\beta - \alpha)t} dt + e^{-j\theta} \int_{-T/2}^{T/2} e^{-j2\pi(\beta + \alpha)t} dt \right].$$

Recalling that

$$\int_{-T/2}^{T/2} e^{j2\pi f t} dt = \frac{\sin(\pi f T)}{\pi f},$$

we obtain

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} \frac{1}{T} \left[e^{j\theta} \frac{\sin(\pi(\beta - \alpha)T)}{\pi(\beta - \alpha)} + e^{-j\theta} \frac{\sin(\pi(\beta + \alpha)T)}{\pi(\beta + \alpha)} \right].$$

Equivalently,

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} [e^{j\theta} \text{sinc}((\beta - \alpha)T) + e^{-j\theta} \text{sinc}((\beta + \alpha)T)],$$

where $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

If $\alpha = \beta$:

$$\hat{x}(\alpha) = \frac{a}{2} e^{j\theta}.$$

If $\alpha = -\beta$:

$$\hat{x}(\alpha) = \frac{a}{2} e^{-j\theta}.$$

Otherwise:

$$\hat{x}(\alpha) = 0.$$

Thus:

$$\hat{x}(\alpha) = \begin{cases} \frac{a}{2} e^{j\theta}, & \alpha = \beta, \\ \frac{a}{2} e^{-j\theta}, & \alpha = -\beta, \\ 0, & \text{otherwise.} \end{cases}$$

This demonstrates that the time-averaged inner product survives only when the test frequency α matches a sinusoidal component β of the signal.

3.1.1 Relation between Strict Periodicity and Cyclostationarity

Every strictly periodic signal is trivially wide-sense cyclostationary. Indeed, if $x(t) = x(t + T)$, then

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t, \tau) = \mathbb{E}[x(t)x^*(t+\tau)] = \mathbb{E}[x(t+T)x^*(t+T+\tau)] = R_x(t+T, \tau).$$

Thus:

$$x(t) \text{ periodic} \implies x(t) \text{ WSC (cyclostationary)}.$$

The converse does not hold: cyclostationary signals need not be strictly periodic pointwise. For example, an amplitude-modulated process $x(t) = m(t) \cos(2\pi f_c t)$ with random data $m(t)$ can be cyclostationary with period $1/f_c$, while not being pointwise periodic.

3.2 Novel combined CSP, DSP approach to creating FPGA cache heirarchies

A function $f : A \rightarrow B$ is *foldable* iff $\exists A_0 \subseteq A, T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

Where A_0 is called the *fundamental domain* & T , the set of transformations mapping from A_0 to A , the *recovery method*.

Motivating this definition with a classic example, consider $\sin(x)$ which has period 2π and can be represented with fundamental domain $\frac{\pi}{4}$ where the recovery method

$$T = \begin{cases} x, & 0 \leq x \leq \frac{\pi}{4}, \\ \pi - x, & \frac{\pi}{4} < x \leq \frac{\pi}{2}, \\ x - \pi, & \frac{\pi}{2} < x \leq \frac{3\pi}{2}, \\ 2\pi - x, & \frac{3\pi}{2} < x \leq 2\pi \end{cases}$$

$$f(x) = \begin{cases} f \circ \tau(x), & 0 \leq x \leq \pi, \\ -f \circ \tau(x), & \pi < x \leq 2\pi \end{cases}$$

Can be used to store effectively $4 \times$ less memory M than the full period **OR** with the same memory footprint but $4 \times$ greater granularity G . In general, the ratio between M & G , $\frac{M}{G}$, is called the *precision-memory gain tradeoff*. $M = \frac{P}{|A_0|}$, P is the period of the function.

Note: it is important that the recovery method $f \circ \tau(x)$ is relatively efficient / portable to an FPGA & that the tradeoff between the computational complexity of the recovery method & the memory gain M is not necessarily linear E.g. $\tau(x) \not\propto M$. A relevant example explored later on [**<empty citation>**] is the function $\arccos(x)$ which requires the computation of $\sqrt{x+1}$ on an FPGA in order to do the lookup on the reduced domain $\arccos : [0, \frac{1}{\sqrt{2}}] \rightarrow [-1, 1]$

Group-Theoretic Interpretation

The set of transformations T can be understood as the action of a *symmetry group* G on the domain A . In the sine example, the relevant group is the *dihedral group* D_4 , which consists of rotations and reflections of a square [**armstrong1988groups**,

gallian2021contemporary]. Each $\tau \in T$ corresponds to an element of D_4 acting on the domain of $\sin(x)$, with reflections accounting for sign changes and rotations corresponding to translations by $\pi/2$.

Thus, the concept of foldability is equivalent to requiring that the function f is invariant under the action of a group G , up to sign or phase factors. The *fundamental domain* A_0 is then a representative region of A under this group action, and folding reduces storage by exploiting orbit representatives of G .

In group-theoretic language, the prior example is invariant under the group action up to a sign factor, i.e.,

$$\sin(x) = \epsilon(\tau) \sin(\tau(x)), \quad \epsilon(\tau) \in \{+1, -1\}, \quad \tau \in G.$$

The fundamental domain A_0 corresponds to a set of orbit representatives of G acting on A , and the recovery method reconstructs the full signal from this reduced domain.

Link to Cyclostationary Signals

Foldable functions are naturally related to cyclostationarity. A cyclostationary signal $x(t)$ has statistics that are periodic with period T .

For a foldable function like $\sin(x)$, the fundamental domain A_0 can be interpreted as a segment over which the first- and second-order statistics repeat under the action of G . In other words, folding exploits the same periodic structure that cyclostationary analysis detects: the mean and autocorrelation within A_0 are sufficient to reconstruct the signal statistics over the entire domain.

Thus, foldability can be seen as a deterministic, structural analogue of cyclostationarity: instead of averaging over time to detect periodic statistics, we use group-theoretic transformations to reconstruct the full signal from a reduced representative domain.

Let $f : A \rightarrow B$ be a foldable function with fundamental domain $A_0 \subseteq A$ and recovery transformations $T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

This implies that the function f is invariant under the group action of G generated by T , and the entire domain A can be reconstructed from A_0 using the transformations in T .

Now, consider the autocorrelation function of f :

$$R_f(x, \Delta) = \mathbb{E}[f(x)f^*(x + \Delta)].$$

Since $f(x) = f(\tau(x))$ for some $\tau \in T$, we have

$$R_f(x, \Delta) = \mathbb{E}[f(\tau(x))f^*(\tau(x + \Delta))].$$

Because $\tau(x), \tau(x + \Delta) \in A_0$ (up to another element of the group), all evaluations of $R_f(x, \Delta)$ are determined entirely by the behavior on A_0 . Therefore,

$$R_f(x, \Delta) = R_f(\tau(x), \Delta), \quad \forall x \in A, \quad \tau \in T, \quad \tau(x) \in A_0.$$

Thus, the fundamental domain A_0 captures all statistical information of the function f , analogous to how the fundamental domain in cyclostationary signal processing captures the periodic statistical properties of a signal.

3.2.1 Procedure to storing arbitrary signals

A procedure to *caching* an arbitrary signal $x(t)$ is the following:

1. Decompose the signal into a series of harmonics (for analysis purposes). In practice, this can be approximated via autocorrelation or spectral analysis rather than explicit FFT computation.
2. Use autocorrelation (ACF) analysis on the fundamental harmonic to estimate the period of a noisy signal within a specified SNR tolerance.
3. Iteratively refine the period estimate using heuristics, e.g., minimize spectral leakage if the period is undershot, or adjust lags using peak prominence methods.
4. Determine if the signal is *foldable* by checking whether there exists a fundamental domain A_0 and a set of transformations T such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \text{ and } f(x) = f \circ \tau(x).$$

This step is limited to signals with known symmetries (e.g., trigonometric functions, piecewise monotonic functions).

5. Construct a recovery method that maps any $x \in A$ to A_0 via the transformations in T , taking into account the desired precision-memory gain tradeoff based on control-theoretic or resource-demand considerations.
6. Implement the recovery method in a synthesizable and efficient form for FPGA deployment, ensuring that the computational complexity of applying $\tau(x)$ does not outweigh the memory savings.

Methods for (1), (2) and (3), previously discussed and implemented, are the subject of extensive optimization effort. Steps (4), (5), and (6), however, are considerably more difficult to achieve in practice on a completely unrestricted problem domain. Indeed, this paper focuses on a subset of signal definitions that can be reliably categorized, as well as on certain elementary signals that are relatively commonplace and often appear as components within more stochastic processes.

3.3 Synthesizable construction of the recovery method, T , for FPGA's

Efficient FPGA Implementation of $\arccos(x)$ Using Quarter-Domain Folding

In previous sections, we discussed foldable functions and their use in memory-efficient lookup. In this section, we consider the case of $\arccos(x)$, which initially seems trivial but actually presents valuable insights for efficient computation on hardware.

Fundamental Domain Recovery

The key identity used for recovery from a quarter-domain representation is:

$$\begin{aligned}\tau(x) &= \frac{1}{\sqrt{2}}\sqrt{x+1}, \\ f(x) &= 2 \cdot \arccos(\tau(x))\end{aligned}\tag{3.1}$$

$$\implies \arccos(x) = 2 \arccos\left(\frac{1}{\sqrt{2}}\sqrt{x+1}\right).\tag{3.2}$$

Applying a third-order Taylor series expansion to $\sqrt{x+1}$ yields:

$$16\sqrt{x+1} \approx x^3 - 2x^2 + 8x + 16.$$

Unlike the arcsin third-order polynomial, this polynomial can be factored over a finite field:

$$(x+1)(x^2+x+1).$$

To recover the original polynomial, we add the remaining series:

$$T_2 = -4x^2 + 7x + 15.$$

Efficient Hardware Computation

Computing the first term:

$$T_1 = (x+1)(x^2+x+1)$$

(**Note** that the second term T_2 can be computed using precomputed values)

$$T_2 = -4x^2 + 7x + 15 = -4x^2 + 8x + 16 - (x+1) = -(x^2 \ll 2) + ((x+1)+1) \ll 3 - (x+1)$$

Which can be implemented as follows:

```
// Begin computation of tau(x) for arccos(x)
// arccos in [0, 1/sqrt(2)], Q2.21 format (24 bit data-width)

// Clock cycle 1: compute shared terms
t1 = x + 1;
x_squared = x * x;

// Clock cycle 2: compute T1
t1 = t1 * (x_squared + t1);

// Clock cycle 2: concurrent with T1, compute T2
t2 = -(x_squared << 2) + ((t1 + 1) << 3) - t1;

// Clock cycle 3: add finite-field-factorization correction term, T2, to T1
tau = t1 + t2;

// Clock cycle 4: right-shift by 4 to get sqrt(x + 1), multiply by 1/sqrt(2) term
localparam inv_sqrt2_q = ...; // 1/sqrt(2) stored in Q2.21 format
tau = (tau >> 4) * inv_sqrt2_q;
```

3.3. SYNTHESIZABLE CONSTRUCTION OF THE RECOVERY METHOD, T , FOR FPGA'S9

```
// Do the other required computations to get T

// Clock cycle 5/6: quantization of tau(x) to indice present in BRAM, LUT lookup
logic [23:0] T;
acos_tau_quantize f (.tau(tau), .f(T), .clk(clk));

// Clock cycle 7: final result
T = T << 1;
```

Resource Analysis and Benefits

The total cost for this third-order implementation of $\tau(x)$:

- 3 multiplications (1 for shared term, 1 for T_1 , 1 for multiplication by $\frac{1}{\sqrt{2}}$)
- 6 additions/subtractions (1 for shared term, 1 for T_1 , 3 for T_2 , 1 for the final sum)
- 4 bit-shifts and 1 bitwise inversion (sign flip)

Advantages:

1. No transcendental operations in the argument apart from the $\frac{1}{\sqrt{2}}$ multiplier, simplifying quantization.
quantization step can be bypassed to save a clock cycle by computing the arccos lut values along a non-linear axis I.e. according to the distribution $\tau(x)$
2. Only a single DSP unit is required for all multiplies if the data width allows single-cycle multiplication.
3. No additional storage for constants required except for $\frac{1}{\sqrt{2}}$.
4. The computation is easily pipelined.
5. Bit-growth is predictable, scaling relative to $\deg(\tau(x))$

Chapter 4

Architectural Design

4.1 Filter design

4.1.1 Procedure for approximating Error statistics on filter kernels

Let f be a monotone function (increasing or decreasing), $f(x) = S = \sum_{i=1}^n g_i(x)$.
Then $f(x)$ “contains” $h(x)$ as a term if $\exists j \in \{1, \dots, n\}$ with $g_j(x) \equiv h(x)$.

Define the error term as the contribution of all other indices:

$$J = \{1, \dots, n\} \setminus \{j\}, \quad \mathbb{E}(x) = \sum_{k \in J} g_k(x).$$

The procedure for finding an error term $\mathbb{E}(x)$ is the following:

$$\begin{aligned} f^{(-n)}(x) \text{ contains } f(x) &\implies \mathbb{E}(x) = f^{(-n)}(x) - \text{sgn } g_j f(x), \\ f^{(n)}(x) \text{ contains } f(x) &\implies \mathbb{E}(x) = f^{(n)}(x) - \text{sgn } g_j f(x). \end{aligned}$$

By the Cauchy formula for repeated integration [**CauchyRepeatedIntegration**] we have:

$$\begin{aligned} f^{(-n)}(x) &= \int_a^x \int_a^{\sigma_1} \dots \int_a^{\sigma_{n-1}} f(\sigma_n) d\sigma_n d\sigma_{n-1} \dots d\sigma_1 \\ &= \frac{1}{(n-1)!} \int_a^x (x - \sigma)^{n-1} f(\sigma) d\sigma. \end{aligned}$$

The *Caputo fractional derivative* is constructed using Cauchy’s formula, bridging definitions of $f^{(-n)}(x)$ and $f^{(n)}(x)$ [**Caputo1967**]:

$${}^C D_a^\nu f(\sigma) = \frac{1}{\Gamma(\lceil \nu \rceil - \nu)} \int_a^\sigma \frac{f^{(\lceil \nu \rceil)}(t)}{(\sigma - t)^{\nu - \lceil \nu \rceil + 1}} dt.$$

Finding error statistics of $\text{sinc}(x)$ for a LPF Design

One of the fundamental results in DSP is that the ideal window function

$$w(t) = \text{sinc}(t)$$

in the time domain corresponds to a perfect rectangular pulse in the frequency domain [OppenheimDSP]. However, as the envelope $\frac{1}{x}$ decays the amplitude of $\text{sinc}(x)$ monotonically, the function has an infinite domain that is impossible to capture discretely. This implies the existence of an associated error term $\mathbb{E}(x)$ that depends on the truncation point x .

Derivative of sinc

$$\begin{aligned} \frac{d}{dx} \text{sinc}(x) &= \frac{\cos(x)}{x} - x^{-2} \sin(x) = \frac{x \cos(x) - \sin(x)}{x^2} \\ &= \frac{1}{x} \cos(x) - \text{sinc}(x). \end{aligned}$$

Thus,

$$f'(x) \text{ contains } f(x) \implies \mathbb{E}(x) = \frac{1}{x} \cos(x).$$

Power series expansion of $\frac{\cos(x)}{x}$

$$\mathbb{E}(x) = \frac{1}{x} \left| \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} \right| = \left| \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n-1}}{(2n)!} \right|.$$

Discrete truncation In practice, we require

$$\mathbb{E}(x, N) \leq T \quad \text{for some } N \in \mathbb{N}, N \geq 1,$$

where $\mathbb{E}(x, N)$ is the partial sum of $\mathbb{E}(x)$, and the choice of $0 < T < f(x)$ determines the minimum N such that $\mathbb{E}(x, N)$ is bounded. Another stipulate is that N is minimal *W.R.T* T .

$$\implies \mathbb{E}(x) = \left| \sum_{n=0}^N \frac{(-1)^n x^{2n-1}}{(2n)!} \right| \leq T.$$

Re-indexing with Kronecker delta Re-indexing to $2N$, we obtain

$$\sum_{k=0}^{2N} \delta_{k \equiv 0 \pmod{2}} \cdot \frac{(-1)^{k/2} x^{k-1}}{k!},$$

which implicitly allows $E : \mathbb{R} \rightarrow \mathbb{R}$ to be re-written as $E : \mathbb{C} \rightarrow \mathbb{C}$ due to the introduction of the *Kronecker delta*[<empty citation>]:

$$\begin{aligned}
\mathbb{E}(x) &= \left| \sum_{k=0}^{2N} \frac{1 + (-1)^k}{2} \cdot \frac{i^k x^{k-1}}{k!} \right| \\
&= \sum_{k=0}^{2N} \delta_{k \equiv 0 \pmod{2}} \frac{(-1)^{k/2} x^{k-1}}{k!} \\
&= \frac{1}{2\pi i} \oint_{|z|=1} \frac{1+z^2}{2z} \sum_{k=0}^{2N} \frac{(-1)^{k/2} x^{k-1} z^{-k}}{k!} dz \\
&= \frac{1}{2\pi i} \oint_{|z|=1} \frac{1+z^2}{2z} \cdot \frac{1}{x} \sum_{k=0}^{2N} \frac{(ix/z)^k}{k!} dz.
\end{aligned}$$

Here, $\Im(\mathbb{E}(x))$ corresponds to the *phase error* and $\Re(\mathbb{E}(x))$ corresponds to the *quantization error*. Extending onto the complex plane requires $T \in \mathbb{C}$, expressed as $a + bi$, where a is the quantization error threshold and b the relative phase error threshold.

Error bounds via alternating series test Because $\cos(x)$ is an alternating series, we can use the alternating series error test [**Rudin1976**]:

$$\mathbb{E}(x) \approx |\cos(x) - \cos_N(x)|.$$

Applying Stirling's approximation [**StirlingApprox**]:

$$\mathbb{E}(x) \leq \frac{|x|^K}{\sqrt{2\pi K}} \left(\frac{K}{e} \right)^K, \quad K = \frac{N+1}{2}.$$

Chapter 5

Control theoretic model for dynamic FPGA resource balancing

Appendix A

Appendix